

DBMS LAB – TASK 4 - CSE 3

SQL JOIN Operations using the Taxation Database

Objective

The objective of this laboratory experiment is to help students understand and apply various SQL JOIN operations to retrieve meaningful information from multiple related tables in a relational database.

By the end of this laboratory experiment, students should be able to:

- Retrieve related information stored across multiple tables.
- Apply different types of SQL JOIN operations.
- Understand how relationships created using Foreign Keys are utilized during data retrieval.
- Write efficient SQL queries using multiple JOIN operations.
- Analyze relational data using practical database queries.

Learning Outcomes

After completing this laboratory experiment, students will be able to:

- Understand the purpose of SQL JOIN operations.
 - Distinguish between different types of JOINS.
 - Retrieve related information from multiple tables.
 - Write queries involving two or more related tables.
 - Apply SQL JOINS to solve real-world database problems.
-

Pre-Requisites (Micro Topics)

Students should revise the following concepts before attending the laboratory.

Module 1 – Relationship between Tables

- Primary Key
 - Foreign Key
 - Referential Integrity
 - One-to-Many Relationship
 - ALTER TABLE
 - Constraints
-

Module 2 – SQL JOIN Operations

- INNER JOIN
 - LEFT OUTER JOIN
 - RIGHT OUTER JOIN
 - FULL OUTER JOIN
 - CROSS JOIN
 - SELF JOIN
-

Module 3 – Multiple Table Queries

- Joining two tables
 - Joining three tables
 - Joining four tables
 - Selecting required columns
 - Using table aliases
-

Introduction

In the previous laboratory, the taxation database was redesigned by establishing relationships between tables using Foreign Key constraints.

The database now consists of the following related tables:

- Taxpayer
- Income_Record
- Income_Category
- Financial_Year

Each income record belongs to

- one taxpayer,
- one income category,
- and one financial year.

Although the data is stored in separate tables to avoid redundancy, users generally require information by combining data from multiple tables.

For example,

- Which taxpayer earned Salary income?
- Which taxpayer received Business income during the financial year 2025–2026?
- Which organization paid the taxpayer?
- Which income category does each taxpayer belong to?

Such information cannot be retrieved from a single table alone.

SQL provides **JOIN** operations to combine rows from two or more related tables based on a common column.

SQL JOIN

A JOIN combines rows from two or more tables using a related column.

The relationship between tables is usually established using a Primary Key and a Foreign Key.

General Syntax

```
✓SELECT column_list  
FROM table1  
JOIN table2  
ON table1.common_column = table2.common_column;
```

Types of SQL JOINS

1. INNER JOIN

General Syntax

```
SELECT columns  
FROM table1  
INNER JOIN table2  
ON table1.column = table2.column;
```

2. LEFT OUTER JOIN

General Syntax

```
SELECT columns  
FROM table1  
LEFT OUTER JOIN table2  
ON table1.column = table2.column;
```

3. RIGHT OUTER JOIN

General Syntax

```
SELECT columns  
FROM table1  
RIGHT OUTER JOIN table2  
ON table1.column = table2.column;
```

4. FULL OUTER JOIN

General Syntax

```
SELECT columns  
FROM table1  
LEFT JOIN table2  
ON table1.column = table2.column
```

UNION

```
SELECT columns  
FROM table1  
RIGHT JOIN table2  
ON table1.column = table2.column;
```

5. CROSS JOIN

General Syntax

```
SELECT columns  
FROM table1  
CROSS JOIN table2;
```

6. SELF JOIN

A SELF JOIN joins a table with itself.

It is useful when comparing rows within the same table.

Use Case

Display taxpayers having the same occupation.

General Syntax

```
SELECT columns  
FROM table_name A  
JOIN table_name B  
ON A.column = B.column;
```

Software Required

- MySQL Server 8.x
 - MySQL Workbench
 - Git
 - GitHub Account
-

Database to be Used

Continue using the database created in the previous laboratory.

taxation_db

Tables Already Available

- Taxpayer
- Income_Category
- Financial_Year
- Income_Record

No new tables need to be created.

The existing Foreign Key relationships established in the previous laboratory will be used to perform all SQL JOIN operations.

Laboratory Activities

Part A – Verify Previous Database

Before starting the experiment,

Open MySQL Workbench.

Select the taxation database.

USE taxation_db;

Display all tables.

SHOW TABLES;

Verify that all four tables are available.

- Taxpayer
- Income_Category
- Financial_Year
- Income_Record

Ensure that the Foreign Key relationships created in the previous laboratory are working correctly before performing JOIN operations.

Part B – SQL JOIN Operations

Level 1 (Understanding)

Task-1 Display every taxpayer along with the income source using an INNER JOIN.

Hint:

Use **INNER JOIN** between **Taxpayer** and **Income_Record** using **taxpayer_id**.

Task-2 Display every taxpayer along with the category of income they earn.

Hint:

Join **Taxpayer**, **Income_Record**, and **Income_Category** tables.

Task-3 Display every income record along with its financial year.

Hint:

Join **Income_Record** with **Financial_Year** using **year_id**.

Task-4 Display the taxpayer name together with the annual income and income amount recorded in the Income_Record table.

Hint:

Use an **INNER JOIN** between **Taxpayer** and **Income_Record**.

Task-5 Display the taxpayer name, income source, category name and financial year for every income record.

Hint:

Join all four tables using their respective Foreign Keys.

Level 2 (Application)

Task-1 Display all taxpayers who earn Salary income along with the organization from which they receive the income.

Hint:

Join **Taxpayer**, **Income_Record**, and **Income_Category**, then filter the category as **Salary**.

Task-2 Display all taxpayers who earn Business income together with their occupation and income source.

Hint:

Join **Taxpayer**, **Income_Record**, and **Income_Category**, then filter the category as **Business**.

Task-3 Display taxpayer details together with the financial year start date and end date.

Hint:

Join **Taxpayer**, **Income_Record**, and **Financial_Year**.

Task-4 Display taxpayer details together with the description of the income category.

Hint:

Join **Taxpayer**, **Income_Record**, and **Income_Category**. Retrieve the **description** column from **Income_Category**.

Task-5 Display complete taxation information by joining Taxpayer, Income_Record, Income_Category, and Financial_Year.

The output should include:

- Taxpayer Name
- PAN Number
- Occupation
- Income Source
- Income Category
- Income Amount
- Financial Year
- Start Date
- End Date

Hint:

Perform an **INNER JOIN** on all four tables using the corresponding Foreign Keys.

Level 3 (Medium to Advanced)

The following activities introduce advanced SQL JOIN operations that are commonly used while developing real-world database applications.

Task-1 Display all taxpayers including those who have not yet submitted any income records.

Hint:

Use a **LEFT OUTER JOIN** between **Taxpayer** and **Income_Record** using `taxpayer_id`.

Task-2 Display all income categories including those that are not associated with any income records.

Hint:

Use a **RIGHT OUTER JOIN** between **Income_Record** and **Income_Category** using `category_id`.

Task-3 Display all taxpayers and all income records, including unmatched records from both tables.

Hint:

MySQL does not support **FULL OUTER JOIN** directly. Simulate it using a combination of **LEFT OUTER JOIN**, **RIGHT OUTER JOIN**, and **UNION**.

Task-4 Generate every possible combination of taxpayers and financial years.

Hint:

Use a **CROSS JOIN** between **Taxpayer** and **Financial_Year**.

Task-5 Display pairs of taxpayers having the same occupation without displaying the same taxpayer twice.

Hint:

Use a **SELF JOIN** on the **Taxpayer** table with different table aliases. Compare the **occupation** column and avoid matching a taxpayer with themselves.

Additional Practice(optional)

Write SQL queries for the following.

Task-6 Display the taxpayer name, PAN number, income source, income category and financial year in a single query.

Hint:

Join all four tables using **INNER JOIN**.

Task-7 Display all taxpayers together with their income category and category description.

Hint:

Join **Taxpayer**, **Income_Record**, and **Income_Category**.

Task-8 Display the income source together with the financial year label.

Hint:

Join **Income_Record** and **Financial_Year**.

Task-9 Display taxpayers whose income belongs to the Business category during the financial year 2025–2026.

Hint:

Join **Taxpayer**, **Income_Record**, **Income_Category**, and **Financial_Year** and apply suitable filtering conditions.

Task-10 Display the complete taxation report containing taxpayer details, income details, category details and financial year details.

Hint:

Retrieve only the required columns by joining all four tables using appropriate **INNER JOIN** statements.

Time-wise Laboratory Plan

Time	Activity
0–20 min	Verify the previous laboratory database and Foreign Key relationships

20–45 min	Practice INNER JOIN queries
45–70 min	Practice joining three and four related tables
70–95 min	Practice LEFT OUTER JOIN and RIGHT OUTER JOIN
95–120 min	Practice FULL OUTER JOIN (using UNION), CROSS JOIN and SELF JOIN
120–150 min	Execute all laboratory tasks and verify outputs
150–180 min	GitHub Submission, Faculty Verification and Discussion

GitHub Submission

Students should create the following folder structure.

StudentID-repo

└─ **Week-4-Lab**

│ └─ **week4_joins.sql**

│ └─ **screenshots.pdf (ignore)**

└─ **report.pdf**

Submission Requirements

Each student must submit:

1. SQL File

`week4_joins.sql`
